

RECURSIVIDAD

¿Qué es?

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip

Caso Base:

- Es el estado simple del problema que se soluciona de forma directa sin realizar nuevas llamadas recursivas.
- Propósito: Evita el desbordamiento de memoria e iteraciones infinitas.

Caso Recursivo

Funcionamiento Interno

- Pila de Llamadas (Call Stack): Estructura de datos LIFO (Last In, First Out) gestionada por el entorno de ejecución para almacenar el estado (variables locales, parámetros) de cada llamada activa.

Clasificación y Características

- Según el modo de invocación:
- Directa: La función `$A$` llama explícitamente a `$A$`.
- Indirecta (Cruzada): La función `$A$` llama a `$B$`, y la función `$B$` llama a `$A$`.
- Según el número de llamadas:
- Simple (Lineal): Realiza una sola llamada recursiva por ejecución.
- Múltiple: Realiza dos o más llamadas recursivas en la misma ejecución (ej. Árboles).

Ventajas y Desventajas

- Elegancia y claridad: Código más conciso, limpio y fácil de leer para problemas intrínsecamente recursivos.
- Simplicidad en estructuras no lineales: Facilita la implementación de recorridos en árboles y grafos.
- Modelado directo: Refleja de forma natural definiciones matemáticas inducentes.

- Es el bloque donde la función se llama a sí misma con parámetros reducidos.
- Propósito: Aproximar paulatinamente el problema hacia el caso base.

- Fase de Crecimiento (Apilado): Cada llamada recursiva añade un marco (frame) a la pila mientras no se alcance el caso base.
- Fase de Desbobinado (Desapilado y Retorno): Al alcanzar el caso base, los resultados parciales se devuelven en orden inverso hasta completar la llamada original.

- Según la posición de la llamada:
 - En la cola (Tail Recursion): La llamada recursiva es la última instrucción ejecutada; permite optimizaciones de memoria por el compilador.
 - No en la cola (Non-tail): Quedan operaciones pendientes después de que retorna la llamada recursiva.

- Desventajas (Palabra de enlace: presenta limitaciones como)
- Consumo de memoria: Alta sobrecarga por el uso continuo de la pila de llamadas (Stack Overflow si la profundidad es excesiva).
 - Rendimiento: Generalmente más lenta que la solución iterativa equivalente debido al costo de gestión de llamadas a funciones.
 - Dificultad de depuración: Rastreo complejo de errores y flujo de control en múltiples niveles.

ALGORITMO RECURSIVO:

Un algoritmo recursivo es aquel que resuelve un problema dividiéndolo en subproblemas más pequeños del mismo tipo, llamándose a sí mismo dentro de su propia ejecución hasta alcanzar una condición de parada.

Para que un algoritmo recursivo funcione adecuadamente y no cree un bucle infinito, debe constar de dos partes obligatorias:

1. Caso Base (o Condición de Parada): Es la condición simple que devuelve un resultado directamente sin realizar más llamadas recursivas. Evita que la función se ejecute indefinidamente.
2. Caso Recursivo: Es la sección del código donde la función se llama a sí misma, pero reduciendo el tamaño del problema para acercarse progresivamente al caso base.

RECURSIVIDAD VS ITERACIÓN

La recursividad y la iteración son dos enfoques fundamentales en programación para repetir un conjunto de instrucciones. La diferencia clave radica en cómo repiten el proceso: la recursividad lo hace mediante llamadas continuas a la misma función, mientras que la iteración lo hace mediante bucles (for, while) que repiten un bloque de código.

Criterio	Recursividad	Iteración
Mecanismo	La función se llama a sí misma.	Utiliza bucles de control (for, while, do-while).
Control de parada	Un Caso Base explícito.	Una condición de término en el bucle.
Uso de Memoria	Alto: Requiere espacio en la pila (Call Stack) para cada llamada.	Bajo: Solo utiliza variables locales en un único espacio de memoria.
Rendimiento	Más lento: Tiene la sobrecarga de crear y destruir marcos de función.	Más rápido: Transiciones directas de instrucciones en la CPU. LegibilidadAlta para problemas jerárquicos o inductivos (árboles, grafos).Alta par
Legibilidad	Alta para problemas jerárquicos o inductivos (árboles, grafos).	Alta para procesos simples o iteraciones sobre colecciones lineales.
Riesgo Principal	Desbordamiento de pila (Stack Overflow).	Bucles infinitos (bloqueo de CPU o freeze del hilo).

Categoría	Casos de Uso	Algoritmos / Ejemplos Concretos
Matemáticas y Series	Cálculo de funciones definidas por inducción.	- Factorial: $n! = n \times (n-1)!$ - Serie de Fibonacci: $F(n) = F(n-1) + F(n-2)$
Estructuras de Datos	Procesamiento de estructuras jerárquicas o auto-referenciadas.	Recorrido de Árboles (Inorden, Preorden, Postorden). Búsqueda en Grafos (DFS - Depth-First Search).
Diseño Algorítmico	Estrategia "Divide y Vencerás" (Divide and Conquer).	- Ordenamiento: Merge Sort, Quick Sort. - Búsqueda: Búsqueda Binaria.
Juegos y Optimización	Exploración del espacio de soluciones (Backtracking).	- Torre de Hanói. - Problema de las N Reinas. - Laberintos.